

Informe NaviRAG Trading- Evaluación Parcial 2

Dato	Información
Asignatura	Ingeniería de Soluciones con Inteligencia Artificial, sección 002D
Evaluación	Evaluación Parcial 2 - Desarrollo de un Agente Funcional
Integrantes	Lucas Huerta y Fabian Morande
Docente	Francisco Andrés Macaya Matas
Fecha	
Repositorio general	https://github.com/Fergchis/NaviRag_Trading
Rama analizada	version-2

Nota de información pendiente

El archivo eval/resultados_ev2.json existe en el repositorio, pero no fue usado como evidencia ni como base de conclusiones, porque sus resultados no están actualizados.

1. Introducción

NaviRAG Trading es una demo educativa orientada a consultar documentos PDF sobre trading mediante lenguaje natural. El sistema usa un agente de IA con recuperación aumentada por generación (RAG), memoria conversacional y una capa de decisión que define el recorrido de cada consulta. Su propósito no es operar en mercados ni entregar recomendaciones financieras, sino apoyar el aprendizaje y la consulta de material educativo.

Este informe resume el diseño, implementación, memoria, recuperación de contexto, planificación, orquestación, repositorio y evidencias pendientes del proyecto para la Evaluación Parcial 2.

2. Contexto y objetivo del proyecto

El caso simulado corresponde a Iwakura Trading Academy, una academia privada que trabaja con PDFs educativos sobre gestión de riesgo, análisis técnico, estrategias, lectura de mercado y fundamentos operativos. La necesidad principal es reducir la dificultad para encontrar conceptos dentro de los documentos y disminuir consultas repetitivas dirigidas a docentes.

El objetivo de NaviRAG Trading es actuar como asistente institucional educativo: recibe preguntas de estudiantes, recupera fragmentos relevantes desde la base documental y genera respuestas explicativas. El alcance está limitado al apoyo académico. No usa datos de mercado en tiempo real, no ejecuta órdenes, no realiza backtesting y no entrega señales de compra o venta.

3. Diseño e implementación del agente

La implementación revisada en la rama version-2 organiza el agente con LangGraph. El archivo agent_app/agent.py define un StateGraph principal con un supervisor y tres agentes especializados: rag_agent, memory_agent y answer_agent. El modelo de chat se configura mediante ChatOpenAI usando GitHub Models, con temperatura 0 para reducir variabilidad en las respuestas.

El supervisor funciona como planificador por turno. Usa una salida estructurada Route para decidir entre cinco rutas: rag_agent, memory_agent, rag_then_answer, answer_agent o FINISH. Esta decisión evita ejecutar todos los componentes en cada consulta y permite seleccionar solo el recorrido necesario.

Las herramientas verificadas en agent_app/tools.py son:

- rag_search(query): herramienta de consulta documental. Genera embedding de la pregunta, ejecuta búsqueda vectorial en MongoDB Atlas y devuelve fragmentos con metadatos como fuente, archivo, chunk_id y score.
- manage_memory: herramienta de escritura de memoria. Permite guardar o actualizar información estable del usuario mediante LangMem.
- search_memory: herramienta de consulta de memoria. Permite recuperar información previamente guardada para el usuario.

La herramienta de escritura implementada corresponde a escritura de memoria, no a generación o edición de documentos. Esta precisión es importante para no atribuir funcionalidades que no aparecen en el código.

4. Memoria y recuperación de contexto

La memoria de corto plazo se implementa con MemorySaver al compilar el grafo principal. En app.py, Streamlit mantiene un thread_id por conversación y lo envía en la configuración de graph.invoke, lo que permite conservar continuidad dentro del mismo hilo mientras el proceso está activo.

La memoria de largo plazo se modela con InMemoryStore y herramientas de LangMem. Las memorias usan el namespace ("agent_memories", "{user_id}"), lo que separa la información por usuario. Sin embargo, esta memoria no es persistente de forma durable.

La recuperación de contexto documental se realiza mediante MongoDB Atlas Vector Search. La ingesta de PDFs está en src/ingesta/ingest.py: los archivos se convierten a texto con MarkItDown, se dividen en fragmentos, se generan embeddings con GitHubModelsEmbeddings y se guardan en MongoDB con metadatos de fuente. Luego, rag_search consulta el índice vectorial y formatea resultados con etiquetas.

5. Planificación y toma de decisiones

El supervisor interpreta la intención del usuario y decide la ruta. Por ejemplo, si la consulta pide fuentes o fragmentos, se usa rag_agent; si pide explicar información de documentos, se usa rag_then_answer; si pide guardar o consultar una preferencia, se usa memory_agent; si es un saludo o una solicitud financiera accionable, se usa answer_agent; y si queda fuera del dominio de trading educativo, se usa FINISH.

La toma de decisiones adaptativa se refuerza con add_conditional_edges. Después de RAG, el flujo puede finalizar o pasar al answer_agent según la ruta inicial. Además, el campo financial_rejection permite activar una respuesta segura cuando el usuario solicita señales, activos específicos o instrucciones financieras accionables. En esos casos, el agente rechaza la solicitud directa y limita la respuesta a una explicación educativa.

6. Orquestación de componentes

La arquitectura general separa interfaz, decisión, recuperación, memoria y respuesta final. Esto mejora la trazabilidad del flujo y permite revisar qué agente fue ejecutado en cada turno.

```
Usuario
-> Interfaz Streamlit (app.py)
-> graph.invoke con thread_id y user_id
-> Supervisor LangGraph
    -> rag_agent -> subgrafo RAG -> rag_search -> MongoDB Atlas Vector Search -> fuentes
    -> memory_agent -> manage_memory / search_memory -> InMemoryStore
    -> answer_agent -> respuesta final educativa / rechazo seguro
    -> FINISH -> salida controlada fuera de dominio
```

El subgrafo RAG también tiene una función específica: el modelo decide llamar rag_search, generate_query reformula la conversación como consulta semántica, ToolNode ejecuta la herramienta y el resultado vuelve al modelo conservando fuentes y metadatos.

7. Pruebas de funcionamiento

El repositorio incluye eval/casos_ev2.json y eval/run_casos_ev2.py para validar rutas como RAG, memoria, respuesta directa, rechazo financiero seguro, salida fuera de dominio, continuidad de hilo y errores inyectados. Como no hay evidencia actualizada disponible, los resultados se dejan pendientes de completar con capturas o salidas nuevas.

Prueba	Objetivo	Resultado o evidencia pendiente
Consulta RAG sobre PDFs	Verificar recuperación documental y uso de fuentes.	Me quede sin tokens :b, adjuntare las pruebas mañana.
Recuperación de contexto	Evidenciar fragmentos, fuente, archivo, chunk_id y score.	
Memoria de usuario	Guardar y luego consultar una preferencia del usuario.	
Respuesta segura	Comprobar rechazo ante solicitud financiera accionable.	
Consulta fuera de dominio	Confirmar salida controlada con ruta FINISH.	
Ejecución por repositorio	Mostrar README, COMO_EJECUTAR.md o ejecución por Streamlit/Docker.	

8. Repositorio, README e instrucciones de ejecución

El repositorio general del proyecto es https://github.com/Fergchis/NaviRag_Trading. Para esta evaluación, la rama relevante es version-2, ya que main no representa la versión actual de la Ev2 y version-1 corresponde al trabajo anterior.

El README.md resume el objetivo, arquitectura, requisitos, pruebas EV2, configuración de LangGraph y límites funcionales. docs/flujo_langgraph_ev2.md documenta nodos, rutas, tools, memoria, flujo Streamlit y matriz de evidencia frente a la pauta.

El archivo COMO_EJECUTAR.md contiene las instrucciones de ejecución en Windows PowerShell. Deja dos alternativas: uso de entorno .venv o uso de Docker. La elección queda a criterio del usuario que ejecute el proyecto. El repositorio también incluye Dockerfile, .env.example, langgraph.json, scripts de ingesta y archivos de evaluación.

9. Conclusión

La arquitectura está dividida en componentes claros, lo que facilita revisar el flujo de trabajo y relacionarlo con los criterios vistos en clase.

Las fortalezas principales son la separación entre supervisor, RAG, memoria y respuesta final y el uso de rutas condicionales. El proyecto cumple con el objetivo de herramienta de la academia inventada “Iwakura trade academy”

10. Referencias

LangChain. (s. f.). Thinking in LangGraph. <https://langchain.com/oss/python/langgraph/thinking-in-langgraph>

Macaya Matas, F. A. (s. f.). Repositorio del docente con materia y ejemplos. GitHub. https://github.com/Foco22/INGENIERIA-DE-SOLUCIONES-CON-INTELIGENCIA-ARTIFICIAL_002D_OLS

Checklist de cumplimiento IE1-IE10

Indicador	Estado	Evidencia usada o sección
IE1	Cumplido	Secciones 3 y 4: rag_search, manage_memory y search_memory.
IE2	Cumplido	Secciones 3, 4 y 6: LangGraph, LangMem, ChatOpenAI, MongoDB Atlas Vector Search.
IE3	Cumplido	Sección 4: MemorySaver e InMemoryStore con namespace por user_id.
IE4	Cumplido	Sección 4: embeddings, búsqueda vectorial y fuentes [FUENTE N].
IE5	Cumplido	Sección 5: Route, supervisor y rutas condicionales.
IE6	Parcial	Sección 7: casos definidos; falta adjuntar capturas/resultados actualizados.
IE7	Cumplido	Secciones 6 y 8: diagrama textual, README y documentación técnica.
IE8	Cumplido	Secciones 3 a 6: justificación de componentes según flujo educativo.
IE9	Parcial	Secciones 6, 7 y 10: diagrama, flujo, referencias; falta evidencia visual final.
IE10	Cumplido	Informe redactado con lenguaje técnico moderado y evidencia del repositorio.